

Escenario

Plataforma distribuida de movilidad urbana bajo demanda **Uber**

Introducción

Construir una plataforma distribuida de movilidad en la que clientes puedan solicitar viajes y conductores habilitados puedan recibir, aceptar o rechazar ofertas. El sistema debe contemplar la cancelación por cliente o conductor, el ciclo completo del viaje y la evolución hacia una arquitectura observable, resiliente y desplegable

Regla tecnológica transversal

El escenario no impone un lenguaje de programación específico. Puede implementarse en Java, TypeScript, C#, Python, Go u otro lenguaje/framework, siempre con aprobación previa de la cátedra. La evaluación se centra en competencias, arquitectura, comunicación, concurrencia, calidad y evidencias, no en una tecnología particular.

Alcance propuesto EA1

ID	Título	Descripción
S2-AE1.1	Actores y vehículos	Clientes, conductores y vehículos Auto/Moto con autenticación/roles básicos.
S2-AE1.2	Disponibilidad	Conductor se conecta, actualiza posición y queda disponible para solicitudes.
S2-AE1.3	Solicitud y despacho básico	Cliente indica origen/destino/tipo; se busca conductor y se envía oferta.
S2-AE1.4	Aceptar, rechazar y cancelar	Conductor acepta/rechaza; cliente puede cancelar; sólo un conductor queda asignado.
S2-AE1.5	Ciclo de viaje	Estados mínimos solicitado/asignado/en curso/completado/cancelado.
S2-AE1.6	Tarifa y comprobante simple	Estimación básica y comprobante de demostración sin pasarela real.

Condición del alcance EA1

La implementación no debe transformarse en el sistema completo. El equipo debe demostrar una vertical funcional coherente. Los restantes módulos pueden quedar como contratos/stubs aprobados, siempre que exista diseño de los límites de responsabilidad y plan de evolución

Actores y conceptos principales

- **Cliente:** persona que solicita un viaje, selecciona Auto/Moto, puede cancelar y calificar.
- **Conductor:** persona habilitada que publica disponibilidad, recibe ofertas y puede aceptar, rechazar o cancelar.
- **Vehículo:** Auto o Moto habilitado para prestar un tipo de servicio.
- **Viaje:** entidad de negocio con ciclo de vida controlado y trazable.
- **Operador/Administrador:** rol de soporte para habilitaciones, bloqueos y consulta operativa.
- **Servicios externos:** mapas/geocoding, pasarela de pagos y canales de notificación, reales o sandbox según la etapa.

Módulos completos

Se presenta la descripción integral de los módulos con el propósito de brindar una visión global de la solución y permitir comprender la aplicación como un sistema completo.

No obstante, para esta instancia deberán considerarse específicamente los lineamientos definidos en el apartado “Alcance de la Versión 1.0”, que determinan las funcionalidades comprometidas para la presente entrega.

La descripción general de los módulos tiene carácter orientativo respecto de la evolución esperada del sistema y permite anticipar el alcance de las versiones posteriores. En función del avance del proyecto y de las decisiones pedagógicas y técnicas adoptadas, los estudiantes, en conjunto con el equipo docente, podrán acordar el intercambio, reordenamiento o reasignación de determinadas funcionalidades entre las versiones 1.0 y 2.0, siempre que se mantengan los objetivos de aprendizaje y el alcance general previsto para la actividad.

Estrategia evolutiva de datos

AE1 puede utilizar una misma instancia física de base de datos para reducir complejidad, siempre separando esquemas y documentando propiedad. Por ejemplo, M2/M3 pueden compartir una instancia “Profiles” y M5/M6 una “Mobility”. En AE2 debe justificarse la separación lógica y queda prohibido consultar directamente tablas de otro servicio. Para AE4 se recomienda database-per-service o una excepción explícitamente fundamentada. Esta progresión permite discutir autonomía, transacciones distribuidas y acoplamiento sin convertir la primera actividad en un ejercicio de infraestructura.

ID	Servicio	Responsabilidad	Persistencia recomendada	Infra/externos
----	----------	-----------------	-----------------------------	----------------

M1	Identidad y Acceso	Registro, autenticación, recuperación de acceso, roles y permisos para clientes, conductores y operadores. Emite/valida tokens y desacopla identidad del resto del dominio.	IdentityDB separada	IdP/OAuth2-OIDC; caché opcional de metadatos
M2	Clientes	Gestiona perfil del cliente, direcciones frecuentes, preferencias, historial resumido y calificaciones emitidas. No accede directamente a las tablas de viajes o pagos.	CustomerDB separada (AE1 puede compartir instancia física, esquema aislado)	Cache Redis opcional de perfil
M3	Conductores y Vehículos	Administra datos del conductor, habilitación, vehículo, tipo Auto/Moto, documentación, estado operativo y calificaciones.	DriverDB separada	Servicio externo opcional de validación documental
M4	Ubicación y Disponibilidad	Recibe posición de conductores, mantiene disponibilidad de corto plazo, consulta conductores cercanos y estima distancia/ETA.	LocationDB opcional para histórico; estado caliente fuera de SQL	Redis obligatorio desde AE2; proveedor de mapas/geocoding
M5	Solicitud y Despacho	Recibe solicitudes de viaje, busca candidatos, envía ofertas con vencimiento y asegura que un viaje sea asignado a un único conductor.	DispatchDB; en AE1 puede compartir MobilityDB con M6, luego justificar separación	Redis para ofertas/TTL; RabbitMQ desde AE2

M6	Viajes y Ciclo de Vida	Administra la máquina de estados del viaje: solicitado, asignado, conductor en camino, arribado, en curso, completado o cancelado. Registra transiciones y reglas de cancelación.	TripDB; en AEI puede compartir MobilityDB con M5	RabbitMQ; validación de QR con M8
M7	Tarifas, Pagos y Liquidaciones	Calcula estimación, registra método de pago, autoriza/captura cobros, aplica cargos de cancelación, reintegros y liquidaciones.	PaymentsDB separada, con historial auditable	Pasarela de pago sandbox/externa; RabbitMQ
M8	Notificaciones, Documentos y Soporte	Envía notificaciones, genera QR de verificación, genera comprobante PDF, permite reenvío y registra tickets de soporte asociados al viaje.	CommunicationsDB separada + almacenamiento de objetos para PDF	RabbitMQ; email/SMS/push; generador QR/PDF

Requerimientos Funcionales

M1 - Identidad y Acceso

ID	Título	Descripción
RF-1.1	Registro de identidad	Permitir registrar clientes y conductores con los datos mínimos definidos por la cátedra.
RF-1.2	Autenticación	Permitir iniciar sesión y obtener una credencial/ticket de acceso para consumir los servicios protegidos.
RF-1.3	Roles y permisos	Diferenciar, como mínimo, Cliente, Conductor y Operador/Administrador.

RF-1.4	Recuperación y revocación	Permitir recuperación de acceso y revocación de credenciales cuando corresponda.
RF-1.5	Integración estándar	La evolución del escenario debe soportar OAuth2/OpenID Connect o un mecanismo equivalente aprobado por la cátedra.

M2 - Clientes

ID	Título	Descripción
RF-2.1	Perfil de cliente	Crear y consultar los datos del cliente y sus preferencias.
RF-2.2	Direcciones frecuentes	Registrar origen/destino favoritos o recientes.
RF-2.3	Historial de viajes	Consultar los viajes del cliente mediante APIs, sin acceso directo a la DB del servicio de viajes.
RF-2.4	Calificación del conductor	Permitir registrar una valoración posterior a un viaje completado.
RF-2.5	Estado de cuenta	Consultar información básica del estado del perfil y bloqueos administrativos.

M3 - Conductores y Vehículos

ID	Título	Descripción
RF-3.1	Perfil de conductor	Crear y consultar datos del conductor y su estado de habilitación.
RF-3.2	Vehículos Auto/Moto	Registrar uno o más vehículos e indicar el tipo de servicio habilitado: Auto y/o Moto.
RF-3.3	Disponibilidad	Permitir al conductor declararse conectado/desconectado y disponible/no disponible.
RF-3.4	Documentación	Registrar metadatos de licencia, seguro y documentación del vehículo.
RF-3.5	Calificación del cliente	Permitir al conductor valorar al cliente al finalizar el viaje.

M4 - Ubicación y Disponibilidad

ID	Título	Descripción
RF-4.1	Actualización de ubicación	Recibir coordenadas del conductor con marca temporal mientras esté conectado.
RF-4.2	Conductores cercanos	Consultar conductores disponibles próximos a un origen y compatibles con Auto/Moto.
RF-4.3	Vencimiento de ubicación	No considerar ubicaciones antiguas; utilizar TTL para evitar conductores fantasma.
RF-4.4	Geocodificación	Resolver direcciones y coordenadas mediante proveedor externo o servicio simulado.
RF-4.5	ETA y distancia	Obtener una estimación de distancia/tiempo para apoyar despacho y tarifa.

M5 - Solicitud y Despacho

ID	Título	Descripción
RF-5.1	Solicitud de viaje	El cliente solicita un viaje indicando origen, destino y tipo Auto/Moto.
RF-5.2	Búsqueda de candidatos	Seleccionar conductores disponibles conforme a proximidad y tipo de vehículo.
RF-5.3	Oferta con vencimiento	Enviar una oferta a uno o más conductores con tiempo máximo de respuesta.
RF-5.4	Aceptar o rechazar	El conductor puede aceptar o rechazar una oferta mientras esté vigente.
RF-5.5	Asignación única	Ante respuestas concurrentes, sólo un conductor puede quedar asignado al viaje.
RF-5.6	Cancelación previa	El cliente puede cancelar la solicitud antes de la asignación o según las reglas definidas.
RF-5.7	Sin disponibilidad	Informar cuando no existan conductores adecuados o expire el proceso de búsqueda.

RF-5.8	Trazabilidad del despacho	Registrar eventos relevantes de oferta, rechazo, timeout, aceptación y cancelación.
---------------	---------------------------	---

M6 - Viajes y Ciclo de Vida

ID	Título	Descripción
RF-6.1	Estados del viaje	Administrar estados válidos y transiciones controladas del viaje.
RF-6.2	Arribo del conductor	Permitir al conductor indicar que llegó al punto de retiro.
RF-6.3	Inicio validado	Iniciar el viaje sólo con una condición de verificación definida; desde AE2 se propone QR de un solo uso.
RF-6.4	Finalización	Finalizar registrando tiempo, distancia y datos necesarios para tarifa/pago.
RF-6.5	Cancelación cliente por	Permitir cancelar por el cliente con motivo, estado y eventual cargo.
RF-6.6	Cancelación conductor por	Permitir cancelar por el conductor con motivo y retorno del cliente al proceso de despacho cuando corresponda.
RF-6.7	Historial transiciones de	Mantener auditoría de las transiciones del viaje sin editar la historia ya registrada.

M7 - Tarifas, Pagos y Liquidaciones

ID	Título	Descripción
RF-7.1	Estimación de tarifa	Calcular una estimación basada en tipo de vehículo, distancia, tiempo u otros parámetros configurables.
RF-7.2	Método de pago	Registrar o seleccionar un medio de pago permitido por el escenario.
RF-7.3	Autorización/captura	Simular o integrar autorización y captura de pago al completar el viaje.
RF-7.4	Cargo de cancelación	Calcular un cargo de cancelación cuando las reglas del viaje lo indiquen.
RF-7.5	Reintegro	Registrar un reintegro total o parcial ante una cancelación válida posterior al cobro.

RF-7.6	Idempotencia de pago	La repetición de una misma orden de cobro no debe generar cobros duplicados.
RF-7.7	Historial financiero	Mantener trazabilidad de operaciones financieras y su estado.

M8 - Notificaciones, Documentos y Soporte

ID	Título	Descripción
RF-8.1	Notificaciones de viaje	Notificar solicitud, asignación, arribo, inicio, cancelación y finalización por uno o más canales.
RF-8.2	QR de verificación	Generar un QR temporal y de un solo uso asociado al viaje para validar el inicio, sin exponer información sensible.
RF-8.3	Comprobante PDF	Generar un PDF posterior a la finalización/pago con datos de viaje, importe, identificadores y fecha.
RF-8.4	Reenvío de comprobante	Permitir volver a solicitar el enlace o envío del comprobante.
RF-8.5	Soporte asociado a viaje	Crear un ticket de soporte asociado a un viaje y registrar su estado.
RF-8.6	Consumo asíncrono	Procesar eventos desde RabbitMQ para evitar que notificaciones/documentos bloqueen el flujo principal.

Requerimientos no funcionales y técnicos

Requerimientos no funcionales

ID	Título	Descripción	Nivel mínimo
RNF-01	Libertad tecnológica con aprobación	El desarrollo puede realizarse en cualquier lenguaje y framework elegido por el alumno o grupo, con aprobación previa de la cátedra. La elección debe justificarse por adecuación al problema, herramientas, interoperabilidad y capacidad de despliegue.	AE1

RNF-02	Arquitectura modular por servicios	Los ocho módulos deben exponerse como servicios/microservicios con responsabilidad explícita. No se permite resolver todo el dominio como una única aplicación monolítica sin justificar una excepción.	AE1
RNF-03	Contenerización OCI	Cada aplicación/servicio y dependencia local debe poder ejecutarse en contenedores mediante Docker, Podman o equivalente aprobado.	AE1
RNF-04	Propiedad de datos	Cada servicio debe documentar si posee DB propia, comparte una instancia física con esquema aislado o comparte una DB por decisión transitoria. Desde AE2 no debe consultar directamente tablas pertenecientes a otro servicio.	AE1→AE2
RNF-05	APIs y contratos	Las comunicaciones síncronas deben usar contratos bien definidos; se recomienda REST/HTTP y documentación OpenAPI. Los errores deben tener formato consistente.	AE1
RNF-06	Cache y estado efímero con Redis	Redis será obligatorio desde AE2 para ubicación/disponibilidad, ofertas con TTL y/o caché de información de alta lectura. Debe evidenciarse uso correcto, expiración e invalidación.	AE2
RNF-07	Mensajería con RabbitMQ	RabbitMQ será obligatorio desde AE2 para al menos dos flujos asíncronos, por ejemplo TripAccepted, TripCompleted, PaymentConfirmed o NotificationRequested.	AE2
RNF-08	Idempotencia	Consumidores de mensajes y operaciones críticas como pago, cancelación o finalización deben tolerar reintentos sin duplicar efectos.	AE2
RNF-09	Consistencia y concurrencia	El sistema debe impedir doble asignación de conductor, doble finalización, doble cobro y transiciones inválidas aunque existan solicitudes concurrentes.	AE1→AE4

RNF-10	Seguridad de transporte y acceso	Usar HTTPS en ambientes publicados; validar entradas; proteger endpoints; gestionar tokens/credenciales de forma segura; no almacenar secretos en el código.	AE2→AE4
RNF-11	Configuración externa	Variables de entorno/secret manager para configuración. No codificar claves, cadenas de conexión o secretos en repositorio.	AE1
RNF-12	Usabilidad y adaptabilidad	Las interfaces propuestas deben ser responsivas y mostrar estados claros para espera, aceptación, cancelación, error y modo degradado.	AE1
RNF-13	Logs y correlación	Logs estructurados con identificador de correlación/tripId para reconstruir un flujo distribuido.	AE2
RNF-14	Resiliencia	Aplicar timeouts, reintentos controlados y Circuit Breaker en dependencias críticas. El fallo de un servicio no debe colgar la interfaz ni producir errores sin tratamiento.	AE4
RNF-15	Alta disponibilidad	Ejecutar al menos dos réplicas de uno o más servicios críticos y demostrar continuidad ante caída de una réplica mediante proxy/balanceador y healthchecks.	AE4
RNF-16	Health checks	Exponer endpoints de salud/liveness/readiness y distinguir dependencia disponible/degradada cuando corresponda.	AE2→AE4
RNF-17	Testing	Incluir pruebas de unidad e integración desde AE1; en AE4 incorporar pruebas de sistema, carga/concurrencia, seguridad y casos de resiliencia.	AE1→AE4
RNF-18	Performance	Definir metas medibles para endpoints/flujo crítico y ejecutar pruebas de carga sobre solicitud/aceptación de viajes.	AE4
RNF-19	Observabilidad	En AE4 incorporar métricas, logs y trazas. Se propone Prometheus/Grafana y trazabilidad de	AE4

		al menos dos recorridos críticos de punta a punta.	
RNF-20	CI/CD	Automatizar build, tests, imágenes y publicación en registry; al menos una entrega debe desplegarse sin intervención manual después de la aprobación del pipeline.	AE4
RNF-21	Publicación	La solución integradora debe estar accesible en Internet o en infraestructura de demostración aprobada por la cátedra.	AE4
RNF-22	Versionado gestión y	Versionado semántico, issues/tareas, commits trazables, releases y README con instrucciones de ejecución.	AE1→AE4
RNF-23	Privacidad auditoría y	Evitar exposición de datos personales en logs, QR o URLs; conservar trazabilidad de operaciones críticas y justificar política de retención.	AE4

Progresión por Actividad de Evaluación

Actividad	Nivel	Alcance técnico-funcional	Foco del Programa 2026
AE1	Base funcional inmersión	Definir los 8 límites de servicio; implementar una vertical funcional mínima que permita registrar actores, publicar disponibilidad, solicitar un viaje Auto/Moto, aceptar/rechazar y cancelar. Comunicación principalmente REST. Contenedores, repositorio, Portafolio y Bitácora. Los servicios no terminados pueden presentarse con contrato + stub aprobado.	Ejes 1 y 2: concurrencia/distribución inicial + desarrollo web.

AE2	Profundización arquitectónica	Trabajo individual sobre una base congelada/provista. Incorporar Redis y RabbitMQ, separar responsabilidades y datos, manejar concurrencia/idempotencia, evolución del flujo, QR/PDF y un servicio externo o sandbox. Documentar decisiones y comparar alternativas.	Eje 3: arquitectura, comunicación, REST, asincronía, SOA e interoperabilidad.
AE4	Integración y producción	Trabajo en pareja. Integrar los 8 servicios y demostrar el recorrido completo con seguridad, testing, resiliencia, HA, CI/CD, observabilidad, OpenAPI y publicación. Incluir análisis de un cuello de botella y propuesta de mejora. Coloquio obligatorio con defensa individual.	Integra Ejes 1–4 y TIFAS.

Alcance mínimo sugerido para AE1

- Definir los ocho servicios y sus contratos, aun cuando algunos se presenten como stub en la primera versión.
- Implementar una vertical funcional con M1–M6 y simulación simple de M7/M8.
- Permitir: registrar actores, publicar disponibilidad, solicitar Auto/Moto, ofrecer viaje, aceptar/rechazar, cancelar y completar un viaje básico.
- Demostrar una situación de concurrencia (por ejemplo, dos conductores intentando aceptar la misma solicitud) y cómo se resuelve.
- Contenerizar y documentar ejecución; usar repositorio, tablero/issues, Portafolio y Bitácora individual.
- No es obligatorio RabbitMQ en AE1; Redis puede introducirse de forma opcional o guiada para ubicación.

10. Alcance mínimo sugerido para AE2

- Incorporar Redis obligatoriamente para ubicación/ofertas/caché según diseño.
- Incorporar RabbitMQ para al menos dos flujos asíncronos y documentar productor, consumidor, evento y estrategia de reintentos.
- Separar o justificar la persistencia por servicio; no realizar lecturas SQL cruzadas entre servicios.
- Resolver idempotencia y al menos un caso de carrera realista: aceptación simultánea, cancelación vs. aceptación o doble cobro.
- Generar QR temporal de validación y PDF de comprobante; ambos deben evitar exposición de datos sensibles.

- Integrar o simular mapas y pasarela de pagos; aplicar timeouts y manejo explícito de error básico.
- Documentar OpenAPI, decisiones y pruebas.

11. Alcance mínimo sugerido para AE4 / TIFAS

- Integrar los ocho servicios y demostrar dos recorridos críticos de punta a punta.
- Contenerización completa, healthchecks y red interna de servicios.
- Redis y RabbitMQ operativos, con evidencia de expiración, mensajes, reintentos e idempotencia.
- Seguridad, controles de autorización y gestión de secretos.
- Alta disponibilidad y prueba de caída de al menos un servicio crítico.
- Resiliencia mediante timeouts, reintentos y Circuit Breaker o patrón equivalente.
- CI/CD, registry, versionado y publicación en entorno aprobado.
- Observabilidad con logs estructurados, métricas y trazas; se propone Prometheus/Grafana u opción equivalente aprobada.
- Pruebas de unidad, integración, sistema, carga/concurrencia y seguridad apropiadas.
- Analizar un cuello de botella cuantificable (por ejemplo, tiempo de asignación o tasa de abandono) y proponer una mejora/MVP.

Evidencias esperadas

- Repositorio Git con historial y releases.
- README reproducible y diagrama de componentes/servicios.
- OpenAPI y catálogo de eventos asíncronos.
- Diagrama de datos/propiedad por servicio y ADRs o decisiones de arquitectura.
- Pruebas y logs/capturas/métricas.
- Portafolio de evolución y Bitácora individual cuando corresponda.
- Demostración de QR y PDF en las etapas donde sean requeridos.

Criterios de aceptación del escenario

- Los requisitos seleccionados para cada AE son trazables a una evidencia concreta.
- Las decisiones de microservicios y datos son justificadas, no sólo dibujadas.
- Se puede ejecutar la solución con instrucciones reproducibles.
- Los flujos concurrentes no generan doble asignación, doble cobro ni estados imposibles.
- Los fallos de servicios externos se tratan explícitamente.
- El escenario puede adaptarse por la cátedra sin cambiar las competencias evaluadas.